

Algorithms and data structures I.

9. Practice

What we know so far:
Comparison-based sorting translates this

Lower bound of comparison based sorting

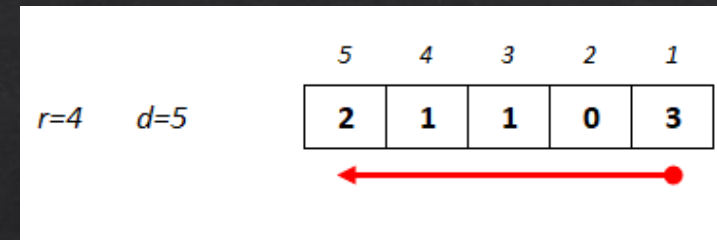
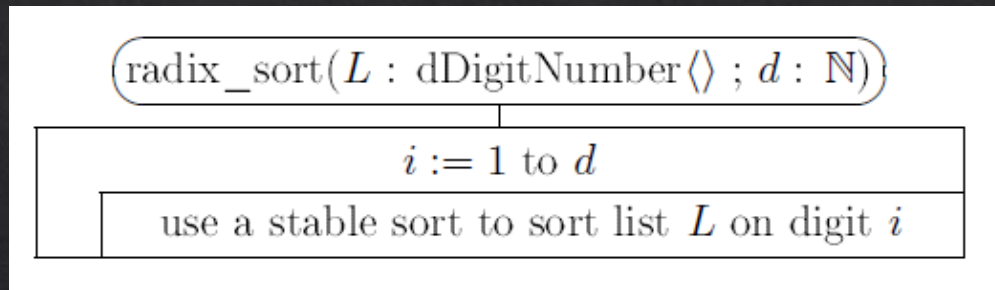
- ◇ 10.1. Theorem. Arbitrary sorting algo: $mT(n) = \Omega(n)$
- ◇ 10.3. Theorem. Any comparison based algo needs $MC(n) = \Omega(n \lg n)$ comparisons.
- ◇ 10.4. Theorem. Any comparison based sorting $MT(n) = \Omega(n \lg n)$.
- ◇ Note that Merge sort and heap sort have $O(n \lg n)$ as an upper bound, and $MT(n) = \Omega(n \lg n)$ lower bound. This means that $MT(n) = \Theta(n \lg n)$.

Radix sort

(Digit based sorting)

Radix sort principle

- ◆ Utilize the fact that our numbers are unsigned r based numerical systems, and have d digits, at most.
- ◆ e.g.: $r=4$, $d=5$: 21103 a possible key.
- ◆ In general:



- ◆ L is an abstract list, `dDigitNumber` is a number defined as above.
- ◆ How is this linear?
- ◆ Why require stability?

Distribution sort

- ◇ Sort the list L
- ◇ We distribute the elements of L into r helper lists in a stable manner (meaning we append at the end).
- ◇ Concatenate the lists one after the other from the back to the front.

$\text{distributing_sort}(L : \mathcal{T}\langle \rangle ; r : \mathbb{N} ; \varphi : \mathcal{T} \rightarrow [0..r))$

$B : \mathcal{T}\langle \rangle[r] //$ array of lists, i.e. bins

$k := 0$ to $r-1$

Let $B[k]$ be empty list $//$ init the array of bins

L is not empty

Remove the first element x of list L

Insert x at the end of $B[\varphi(x)] //$ stable distribution

$k := r-1$ downto 0

$L := B[k] + L //$ append $B[k]$ before L

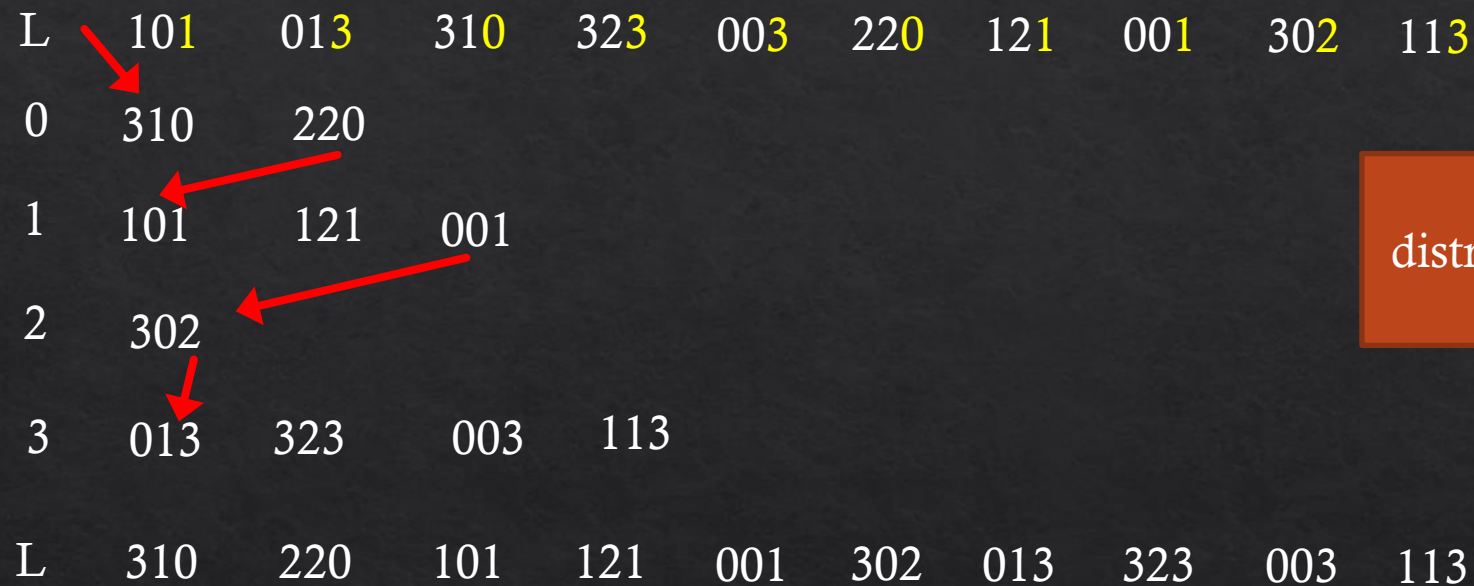
Distribution sort as a radix sort

- ◆ The input has to be a linked list!
- ◆ We iterate over the elements based on their radix, starting from the least significant digit.
- ◆ In case of numeral system we need $0..r-1$ list, distribute elements into their corresponding list by appending at the very end (we might need a pointer to pointing at the end of the list)
- ◆ Then append the lists one after the other.
- ◆ Example for the list below: ($r=4$, $d=3$):

$L = \langle 101, 013, 310, 323, 003, 220, 121, 001, 302, 113 \rangle$

I. - Distribution sort (for lists)

$L = \langle 101, 013, 310, 323, 003, 220, 121, 001, 302, 113 \rangle$



1. iteration:
distribute elements based on their
3rd (last) digit.

$L = \langle 310, 220, 101, 121, 001, 302, 013, 323, 003, 113 \rangle$

$L = \langle 310, 220, 101, 121, 001, 302, 013, 323, 003, 113 \rangle$

L	310	220	101	121	001	302	013	323	003	113
0	101	001	302	003						
1	310	013	113							
2	220	121	323							
3										

2. iteration:
Sorting based on the second digit
from the right.

L 101 001 302 003 310 013 113 220 121 323

$L = \langle 101, 001, 302, 003, 310, 013, 113, 220, 121, 323 \rangle$

$L = \langle 101, 001, 302, 003, 310, 013, 113, 220, 121, 323 \rangle$

L	101	001	302	003	310	013	113	220	121	323
0	001	003	013							
1	101	113	121							
2	220									
3	302	310	323							

3. iteration:
Last digit from the right

L	001	003	013	101	113	121	220	302	310	323
---	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

$L = \langle 001, 003, 013, 101, 113, 121, 220, 302, 310, 323 \rangle$

Implementation for C2Ls

$\text{radix_sort}(L : E2^* ; d, r : \mathbb{N})$

$BinHead : E2[r]$ // the headers of the lists representing the bins

$B : E2^*[r]$ // pointers to the headers

$i := 0$ to $r - 1$

$B[i] := \&BinHead[i]$ // Initialize the i th pointer.

$i := 1$ to d

$\text{distribute}(L, i, B)$ // Distribute L on the i th digits of keys.

$\text{gather}(B, L)$ // Gather form the bins back into L

$\text{gather}(B : E2^*[r] ; L : E2^*)$

$i := 0$ to $r - 1$

$\text{append}(L, B[i])$ // add to the end of L the elements form $B[i]$

$\text{distribute}(L : E2^* ; i : \mathbb{N} ; B : E2^*[r])$

$L \rightarrow next \neq L$

$p := L \rightarrow next ; \text{out}(p)$

$\text{precede}(p, B[\text{digit}(i, r, p \rightarrow key)])$

$\text{append}(L, Bi : E2^*)$

$Bi \rightarrow next \neq Bi$

$p := L \rightarrow prev ; q := Bi \rightarrow next ; r := Bi \rightarrow prev$

$p \rightarrow next := q ; q \rightarrow prev := p$

$r \rightarrow next := L ; L \rightarrow prev := r$

$Bi \rightarrow next := Bi ; Bi \rightarrow prev := Bi$

SKIP

Implementation for S1Ls

radix_sort(&L:E1*,d,r:N)

First:E1*[r]; Last:E1*[r]

i = 1 to d

i = 0 to r-1

First[i]:=Last[i]:=0

distribute(L,i,First,Last)

gather(L,First,Last)

distribute(&L:E1*, i:N, First:E1*[r]; Last:E1*[r])

L ≠ 0

p:=L; L:=L->next

k:=digit(i,r,p->key)

First[k] = 0

First[k]:=p

Last[k]->next:=p

Last[k]:=p; p->next:=0

gather(&L:E1*,First:E1*[r],Last:E1*[r])

L := 0

i = r-1 downto 0

First[i] ≠ 0

Last[i]->next := L

L := First[i]

skip

II. - Counting sort

A) – only one digit case

Counting sort

- ◇ Input: A array
- ◇ Output: B array
- ◇ Contains 0..r-1 integers (or can be mapped into them)

$\text{counting_sort}(A, B : \mathcal{T}[n] ; r : \mathbb{N} ; \varphi : \mathcal{T} \rightarrow [0..r))$

$C : \mathbb{N}[r] //$ counter array
$k := 0$ to $r-1$
$C[k] := 0 //$ init the counter array
$i := 0$ to $n-1$
$C[\varphi(A[i])]++ //$ count the items with the given key
$k := 1$ to $r-1$
$C[k] += C[k-1] //$ $C[k] :=$ the number of items with key $\leq k$
$i := n-1$ downto 0
$k := \varphi(A[i]) //$ $k :=$ the key of $A[i]$
$C[k] -- //$ The next one with key k must be put before $A[i]$ where
$B[C[k]] := A[i] //$ Let $A[i]$ be the last of the {unprocessed items with key k }

- ◇ First step:
- ◇ Count the occurrences $C[0..r-1]$ of the numbers
- ◇ Cumulate the result.

$\text{counting_sort}(A, B : \mathcal{T}[n] ; r : \mathbb{N} ; \varphi : \mathcal{T} \rightarrow [0..r))$

$C : \mathbb{N}[r]$ // counter array

$k := 0$ to $r-1$

$C[k] := 0$ // init the counter array

$i := 0$ to $n-1$

$C[\varphi(A[i])]++$ // count the items with the given key

$k := 1$ to $r-1$

$C[k] += C[k-1]$ // $C[k]$:= the number of items with key $\leq k$

$i := n-1$ downto 0

$k := \varphi(A[i])$ // k := the key of $A[i]$

$C[k]--$ // The next one with key k must be put before $A[i]$ where

$B[C[k]] := A[i]$ // Let $A[i]$ be the last of the $\{\text{unprocessed items with key } k\}$

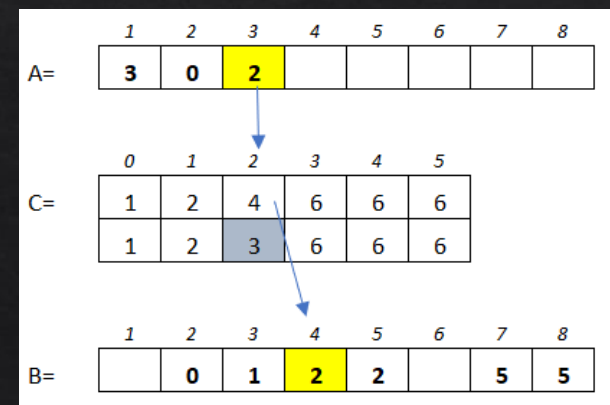
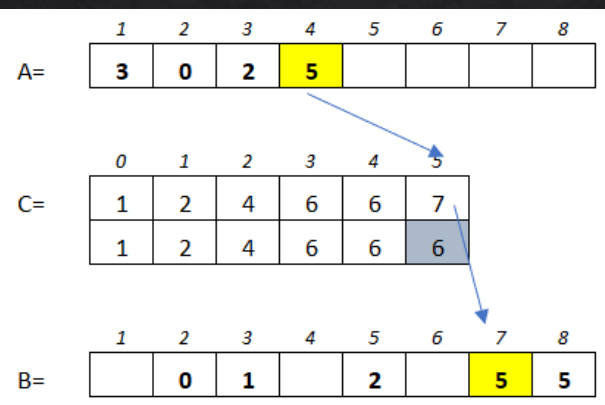
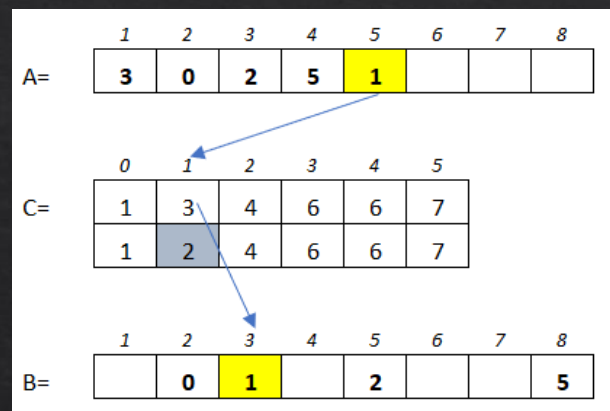
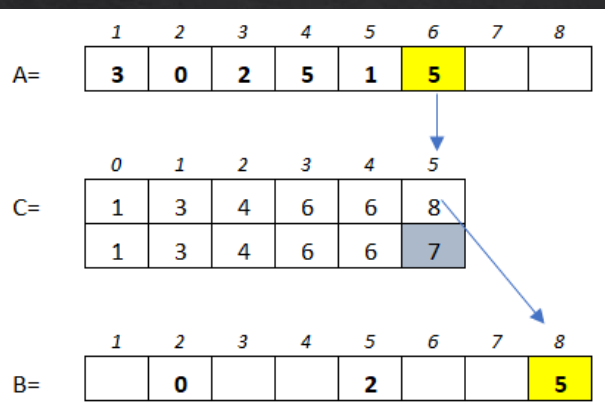
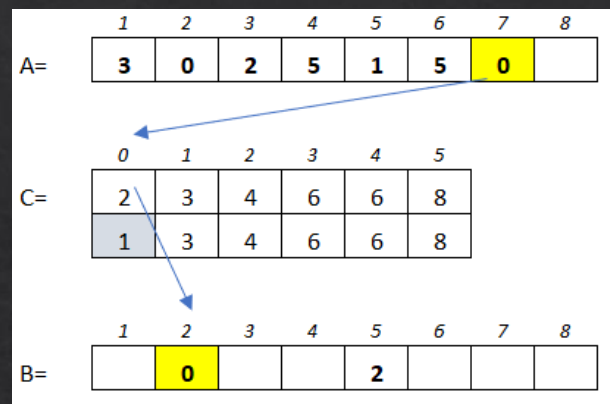
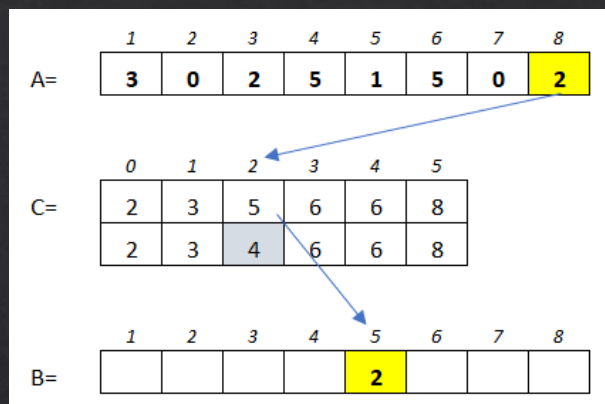
	1	2	3	4	5	6	7	8
A=	3	0	2	5	1	5	0	2

	0	1	2	3	4	5
C=	2	1	2	1	0	2

$C[i]$ indicates how many times i occurs

	0	1	2	3	4	5
C=	2	3	5	6	6	8

Cumulative results



Second step: Move the elements into their proper slot

$\text{counting_sort}(A, B : \mathcal{T}[n] ; r : \mathbb{N} ; \varphi : \mathcal{T} \rightarrow [0..r))$

```

C :  $\mathbb{N}[r]$  // counter array
k := 0 to r-1
  C[k] := 0 // init the counter array
i := 0 to n-1
  C[ $\varphi(A[i])$ ]++ // count the items with the given key
k := 1 to r-1
  C[k] += C[k-1] // C[k] := the number of items with key  $\leq k$ 
i := n-1 downto 0
  k :=  $\varphi(A[i])$  // k := the key of A[i]
  C[k]-- // The next one with key k must be put before A[i] where
  B[C[k]] := A[i] // Let A[i] be the last of the {unprocessed items with key k}

```

Iterating over A backwards and putting elements into B results in a stable sort!

Note: On the images the arrays are indexed from 0 but that is not correct, as you can see in the last two instructions in the stuctugram we first subtract, and then use it as an index.

II. - Counting sort

B) – multi digit case

Radix sort on arrays using counting sort

- ◆ Keys as in distribution sort (r, d)
- ◆ We will iterate on the digis from right to left as well, but using counting sort
- ◆ We will use two arrays moving them from one into the other
- ◆ Example: $(r=4, d=2)$:
 $\langle 11, 20, 10, 23, 21, 30 \rangle$

$A = \langle 11, 20, 10, 23, 21, 30 \rangle$

Counting sort on the second digit

		11	20	10	23	21	30			30	21	23	10	20	11
0	0		1	2			3	3	3	2			1	0	
1	0	1				2		2	5		4				3
2	0							0	5						
3	0				1			1	6			5			

Once inserting into it's place we subtract one from the cumulative occurances

B=	1	2	3	4	5	6
	20	10	30	11	21	23

$B = \langle 20, 10, 30, 11, 21, 23 \rangle$

$B = \langle 20, 10, 30, 11, 21, 23 \rangle$

Counting sort on the first digit

		20	10	30	11	21	23			23	21	11	30	10	20
0	0							0	0						
1	0		1		2			2	2			1		0	
2	0	1				2	3	3	5	4	3				2
3	0			1				1	6				5		

A =	1	2	3	4	5	6
	10	11	20	21	23	30

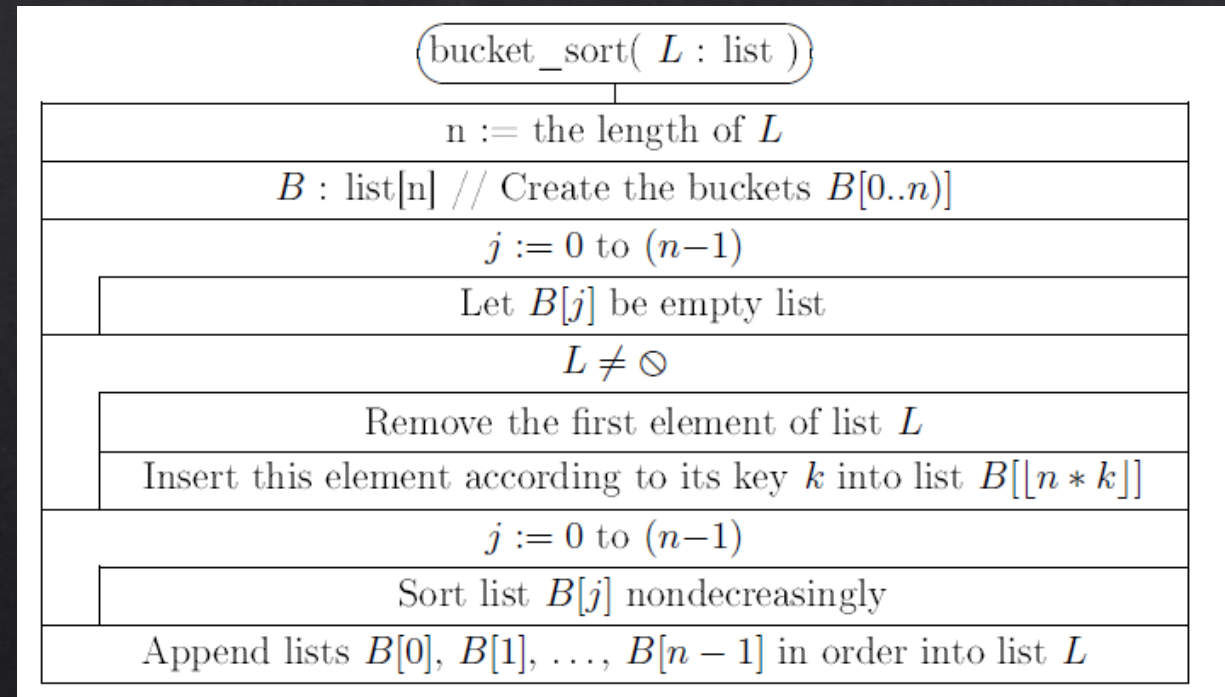
$A = \langle 10, 11, 20, 21, 23, 30 \rangle$

III. - Bucket sort

A) – „Simple”

Bucket sort

- ◆ This time the pre-condition is to have keys from the interval $[0; 1)$.
- ◆ Having n keys to be sorted, create n buckets as well: $B[0..n-1]$
- ◆ Calculate $\lfloor n * k \rfloor$ (bucked index), and insert key into that bucked.
- ◆ Sort the buckets and append one after the other.
- ◆ Of coure now the time complexity is based on how we sort the buckets. The length of the buckets however depend on the distribution of our initial keys. If the keys represent a normal distribution we can have an upper bound in their length and ideally they should be very short lists. See more details in the lecture notes.



Bucket sort demonstration

Input: $\langle 0.79, 0.13, 0.16, 0.64, 0.39, 0.20, 0.89, 0.53, 0.71, 0.42 \rangle$

	After insertion		After sort	For lists containing multiple elements we insert into the front.
0	$\langle \rangle$		$\langle \rangle$	
1	$\langle 0.13 \rangle$	$\langle 0.16, 0.13 \rangle$	$\langle 0.13, 0.16 \rangle$	
2	$\langle 0.20 \rangle$		$\langle 0.20 \rangle$	
3	$\langle 0.39 \rangle$		$\langle 0.39 \rangle$	
4	$\langle 0.42 \rangle$		$\langle 0.42 \rangle$	
5	$\langle 0.53 \rangle$		$\langle 0.53 \rangle$	
6	$\langle 0.64 \rangle$		$\langle 0.64 \rangle$	
7	$\langle 0.79 \rangle$	$\langle 0.71, 0.79 \rangle$	$\langle 0.71, 0.79 \rangle$	
8	$\langle 0.89 \rangle$		$\langle 0.89 \rangle$	
9				

Output: $\langle 0.13, 0.16, 0.20, 0.39, 0.42, 0.53, 0.64, 0.71, 0.79, 0.89 \rangle$

Bucket sort on S1L

bucket_sort(&L : E1*)

n := length(L)			
Z : E1*[n] //edények listáinak első elemére mutató pointerok tömbje			
j = 0 to n-1			
<table> <tr> <td>Z[j] := 0;</td></tr> </table>	Z[j] := 0;		
Z[j] := 0;			
L ≠ 0			
<table> <tr> <td>p, L := L, L->next</td></tr> <tr> <td>j := [n * p->key]</td></tr> <tr> <td>p->next:=Z[j] Z[j]:=p //lista elejére fűzünk</td></tr> </table>	p, L := L, L->next	j := [n * p->key]	p->next:=Z[j] Z[j]:=p //lista elejére fűzünk
p, L := L, L->next			
j := [n * p->key]			
p->next:=Z[j] Z[j]:=p //lista elejére fűzünk			
j = 0 to n-1			
<table> <tr> <td>sort(Z[j])</td></tr> </table>	sort(Z[j])		
sort(Z[j])			
appendLists(L,Z)			

- ◇ Length(L:E1*):N Counts the elements (linear time)
- ◇ Using merge sort: merge(&L:E1*) (n logn time, if upper bound is not accounted)
- ◇ appendLists(&L:E1*, Z:E1*[n]) linear time.

III. - Bucket sort

B) – Binary radix sorting

Binary radix sorting

- Because of the nature of binary representation, **binary radix sorts** have taken on an important role.
- Let's take a look at two variants:
 - A "**binary radix back**" (LSD) sort that alternates between two arrays.
 - An **in-place "binary radix forward"** (MSD) sort.
- These algorithms are provided only as curiosities; **they will not be included in the exam.**

(1) "Binary Radix Back" (2-Array Version)

- We process the bit positions "backwards," moving from the **least significant bit** (LSB) toward the **most significant bit** (MSB). It's conceptually similar to a linked-list radix sort, but it operates within arrays.
- Since we are sorting binary numbers, we only need to split the data into two categories, meaning we only need **two "buckets"** for distribution.
- **How it works:**
 - **The 0-bucket:** These elements are placed at the beginning of the array, starting from index 1 and moving upward.
 - **The 1-bucket:** These elements are placed at the end of the array, starting from index n and moving downward.
- We repeat this for each bit position from right to left. To ensure **stability** (meaning elements with the same value keep their relative order), we must always process the 0-bucket first (from 1 increasing) followed by the 1-bucket (from n decreasing).

Representation

- Let's illustrate the sorting on the array **A** = <101, 001, 011, 110, 010, 111, 000>.
- We process the elements of array **A**.
- If it ends in **1**, it is placed into **bucket 1**. Bucket 1 starts at the "end" of array **B** and grows toward the middle of **B**. (Blue arrow.)
- If it ends in **zero**, it is placed into **bucket 0**. Bucket 0 starts at the beginning of array **B** and grows toward the middle. (Yellow arrow.)

A	B	B	B	B	B	B	B
101				110	110	110	110
001					010	010	010
011							000
110						111	111
010			011	011	011	011	011
111		001	001	001	001	001	001
000	101	101	101	101	101	101	101



Continuing with the **second bit**.

- We sort from B back into A.
- First, we process the 0-bucket, reading it in the 1..n direction.
- Afterward, we process the 1-bucket, reading it in the n..1 direction.
- The buckets are now created in array A, similarly to the previous pass.

	B	A	A	A	A	A	A	A
1.	110			000	000	000	000	000
	010				101	101	101	101
	000					001	001	001
2	111							111
	011						011	011
	001		010	010	010	010	010	010
	101	110	110	110	110	110	110	110

Continuing with the third bit (the most significant bit).

- Continuing with the **third bit (the most significant bit)**.
- We sort from A into B. First, we process the 0-bucket, reading it in the 1..n direction.
- Afterward, we process the 1-bucket, reading it in the n..1 direction.
- The buckets are now created in array B again, similarly to the previous pass.

	A	B	B	B	B	B	B	B
1	000	000	000	000	000	000	000	000
	101			001	001	001	001	001
	001					010	010	010
2	111						011	011
	011							111
	010				110	110	110	110
	110		101	101	101	101	101	101

Final Pass

- It differs from the previous passes.
- We sort from B into A.
- Copy the 0-bucket into A.
- Reverse the content of the 1-bucket: reading it in the n..1 direction, move it into A behind the 0-bucket.
- With this, the sorted array is produced.

	B	A	A	A	A	A	A	A
1	000	000	000	000	000	000	000	000
	001		001	001	001	001	001	001
	010			010	010	010	010	010
	011				011	011	011	011
2	111					101	101	101
	110						110	110
	101							111

(2) - "Binary Radix Forward" Sort (1-Array Version)

- The binary radix forward sort uses a single array and processes bit positions from the **most significant** (largest) to the **least significant** (smallest), swapping elements within a single pass.
- Non-negative binary numbers can be sorted via the radix method by starting with the most significant bit and using swaps to ensure that numbers starting with **0** are at the front, while those starting with **1** are in the second half of the array.
- To do this, we start from the beginning of the array to find a number starting with a **1** bit, and simultaneously start from the end of the array to find a number starting with a **0** bit. If the two resulting indices are different, we **swap** the elements.
- We continue this process until the two indices meet. At this point, the front of the array will consist entirely of numbers starting with a **0** bit, and the end will consist of those starting with a **1** bit.
- We then **recursively** apply the same process to both the "zero" and "one" buckets.

Representation

- Demonstrate the operation of the algorithm on the array **<101, 001, 011, 110, 010, 111, 000>**:

101	000	000	E0	000	E00	000	000	000
001	001	001		001		001	001	001
011	011	011		011	E01	011	011	010
110	110	010		010		010	010	011
010	010	110	E1	110	E10	101	101	101
111	111	111		111	E11	111	111	110
000	101	101		101		110	110	111
<i>csere</i>	<i>csere</i>			<i>csere</i>			<i>csere</i>	<i>kész</i>

Structogram

- The array segment to be sorted: $ah..fh$ (from the starting index to the finishing index).
- We sort based on the i -th bit. The index $i0$ starts first, moving from ah , and stops when it encounters a 1 bit.
- Index $i1$ also starts, stepping downward, and stops when it encounters a 0 bit.
- If the two indices have not met, a swap occurs, and then $i0$ and $i1$ start moving again from positions shifted by one relative to the swap.

